

Ứng dụng tác tử AI trong lập trình nhúng: kiến trúc, những việc làm được và kết quả trên ba bo mạch thật

Vũ Trí Công, Nguyễn Trung Hiếu

Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, Việt Nam

Email: CongVT.B24CHDT005@stu.ptit.edu.vn, hieunt@ptit.edu.vn

Tóm tắt—Bài báo trình bày EIDE, một tác tử (agent) dùng mô hình ngôn ngữ lớn để làm phần mềm nhúng cùng kỹ sư, và trả lời câu hỏi: tác tử này làm được những việc gì, và kết quả có tin được không. Về kiến trúc, tác tử là một vòng lặp do mô hình điều khiển, bao quanh bởi ba lớp chặn viết bằng mã (luật, cửa duyệt, hộp cát), với 127 công cụ, 11 cửa duyệt cho các thao tác có hậu quả, và một số ghi việc móc bấm để mọi con số đều lần về được nguồn. Về năng lực, tác tử đọc tài liệu kỹ thuật, dựng bản đồ mạch, viết và dịch mã, mô phỏng theo tiêu chí nêu trước, nạp và đọc ngược bo thật, tổng hợp và nạp FPGA, tự viết công cụ mới khi thiếu. Tác tử đã làm ba việc thật, cùng một mô hình gemini-3.8-flash, đo bằng số ghi việc chứ không bằng lời kể: tự viết nhân hệ điều hành thời gian thực thay FreeRTOS trên STM32F469I-DISCO (1 211 dòng, nhịp đo thật 1 003 Hz); phần mềm cho robot hai bánh tự cân bằng trên ATmega328P (đứng được, làm hai lần, 7,4 giờ rồi 4,53 giờ); và lõi RISC-V PicoRV32 trên FPGA Tang Nano 20K với 96 ô đo nhân ma trận trên silicon, lệch 0 % so với mô phỏng. Kết quả cho thấy tác tử giúp nhiều nhất ở chỗ làm đủ mọi phép đo mà người thường bỏ, và ở chỗ đi được vào vùng chưa có công cụ; nhưng nó không tự biết nó sai, và chất lượng đề bài do người viết quyết định phần lớn chất lượng kết quả.

Từ khóa—tác tử AI, mô hình ngôn ngữ lớn, lập trình nhúng, hệ điều hành thời gian thực, FPGA, RISC-V, kiểm soát của con người.

I. GIỚI THIỆU

Quyết định số 21/2026/QĐ-TTg xếp trí tuệ nhân tạo và công nghệ bán dẫn vào danh mục công nghệ chiến lược [1]. Lập trình nhúng nằm ở giao điểm của hai hướng đó: phần mềm chạy trên vi điều khiển và trên chip lập trình được (FPGA). Mô hình ngôn ngữ lớn (LLM) đã được thử cho việc này. Englhardt và cộng sự [2] cho thấy LLM viết và sửa được mã cho vi điều khiển nhưng hay sai thông số phần cứng và cần người kiểm. Li và cộng sự [3] bổ sung kỹ năng chuyên gia cho tác tử lập trình nhúng và IoT rồi đo trên bộ bài tập. Zhong và cộng sự [4] chỉ ra một rủi ro riêng: khi được chấm bằng bộ kiểm, mô hình có xu hướng tìm cách làm cho bộ kiểm xanh thay vì làm cho mã đúng.

Các nghiên cứu trên chủ yếu đo mô hình làm được gì trên bài tập. Câu hỏi của kỹ sư thì khác: nếu giao cho một tác tử một việc nhúng thật, từ tài liệu phần cứng tới bo mạch chạy được, thì nó làm được tới đâu, sai ở đâu, và phải tổ chức thế nào để kết quả tin được. Bài báo này trả lời bằng ba việc thật, trên ba bo khác nhau, với cùng một tác tử và cùng một mô hình.

Tác tử đó là EIDE, do nhóm tác giả xây dựng. Kiến trúc của nó (vòng lặp có kiểm soát) được so sánh với các kiến trúc khác trong một bài riêng [5]; bài này tóm tắt kiến trúc ở mức

cần để hiểu kết quả, rồi tập trung vào năng lực và kết quả đo. Mã nguồn, số ghi việc và nhật ký của cả ba việc được công bố [6] để người đọc kiểm lại.

Đóng góp của bài báo: (1) mô tả một tác tử kỹ sư nhúng hoàn chỉnh, với 127 công cụ và các cơ chế bằng mã giữ cho nó không nói sai; (2) kết quả ba việc thật đo trên bo: nhân hệ điều hành thời gian thực tự viết, robot tự cân bằng, lõi RISC-V trên FPGA; (3) phân tích tác tử giúp được ở đâu, không giúp được ở đâu, và điều gì quyết định khoảng chênh đó.

II. KIẾN TRÚC TÁC TỬ

A. Hai phần, một cửa vào

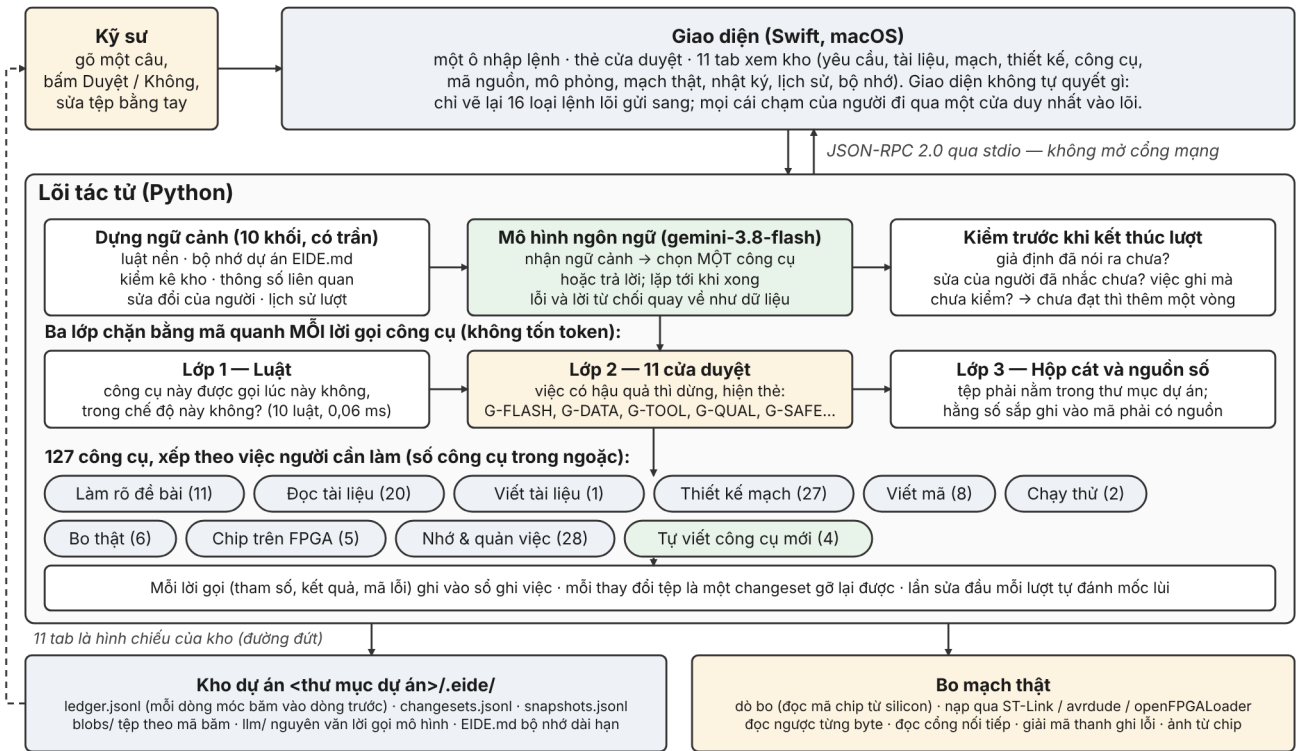
EIDE gồm một giao diện viết bằng Swift trên macOS và một lõi tác tử viết bằng Python (Hình 1). Hai phần nói với nhau bằng JSON-RPC 2.0 qua luồng vào-ra chuẩn; không có cổng mạng nào được mở. Giao diện không tự quyết điều gì: mọi thứ hiện trên màn hình đều do lõi gửi sang qua 16 loại lệnh, và mọi thao tác của người (gõ câu, bấm nút trên thẻ, sửa tệp) đi qua đúng một cửa vào lõi. Nhờ vậy, mọi hành vi có thể kiểm thử bằng cách gửi vào cửa đó, và giao diện có thể thay mà không đổi lõi.

Toàn bộ dữ liệu của một dự án nằm trong thư mục .eide/: số ghi việc (mỗi dòng móc mã bấm vào dòng trước), các lần sửa, bản chốt, nội dung tệp theo mã bấm, và nguyên văn từng lời gọi mô hình. Mười một tab của giao diện chỉ là hình chiếu của kho này; đem thư mục sang máy khác là dựng lại được cả phiên.

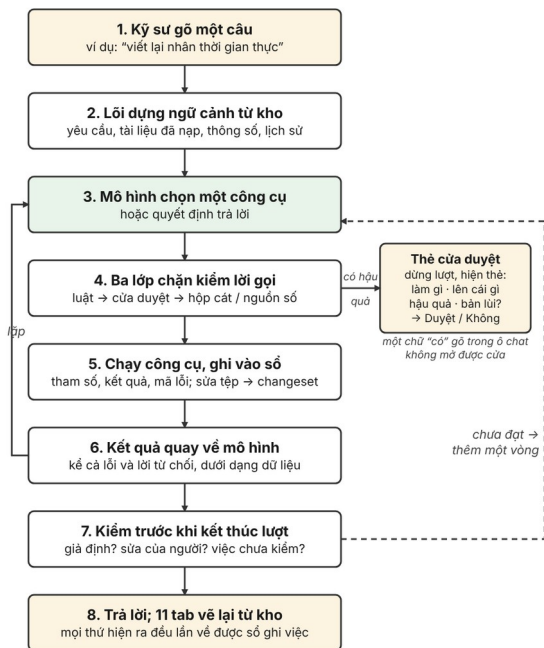
B. Vòng lặp và ba lớp chặn

Một lượt làm việc đi như Hình 2. Lõi dừng ngữ cảnh từ kho (luật nền, bộ nhớ dự án EIDE.md, bảng kiểm kê sinh từ kho để mô hình không mô tả dự án theo trí nhớ, thông số, sửa đổi của người, lịch sử lượt), chia thành 10 khối có trần riêng. Mô hình chọn một công cụ, nhận kết quả, rồi chọn tiếp cho tới khi trả lời; lỗi và lời từ chối quay về dưới dạng dữ liệu có mã lỗi và gợi ý, nên mô hình đổi hướng thay vì dừng.

Điều giữ cho vòng lặp này không làm bậy không nằm trong lời nhắc, mà nằm trong ba lớp mã bao quanh mỗi lời gọi công cụ. Lớp 1 là bộ luật: công cụ này có được gọi lúc này, trong chế độ này không (10 luật, xử lý trong 0,06 ms, chặn trước khi tốn token). Lớp 2 là cửa duyệt: nếu việc có hậu quả thì dừng lượt, hiện một thẻ nói rõ sẽ làm gì, lên cái gì, hậu quả là gì, và chờ người bấm Duyệt hoặc Không. Lớp 3 là hộp cát: tệp phải nằm trong thư mục dự án, và hằng số sắp ghi vào mã phải có nguồn (kho thông số, bộ nhớ dự án, nguồn lời gọi khai, hoặc lời người dùng). Trước khi kết thúc lượt còn một bước kiểm: giả định đã nói ra chưa, sửa đổi của người đã nhắc chưa, có việc ghi mà chưa kiểm không; chưa đạt thì thêm một vòng.



Hình 1. Kiến trúc tác tử EIDE: giao diện chỉ về lại thứ lỗi gửi; lỗi là một vòng lặp do mô hình điều khiển, mỗi lời gọi công cụ đi qua ba lớp chặn bằng mã; mọi việc ghi vào kho `.eide/` của dự án; 11 tab là hình chiếu của kho.



Hình 2. Một lượt làm việc. Mô hình chọn lại ở mỗi bước dựa trên kết quả thật; những việc không được làm do mã quyết, và việc có hậu quả dừng lại chờ người.

C. Mười một cửa duyệt và ba cách gỡ lại

Mỗi loại việc có hậu quả có một cửa riêng: ghi hay nâng mức tin một con số, chốt một cách làm, ghi hay xóa tệp, nạp chip, gỡ lại hay đổi nhánh, chạy lệnh hệ thống, đổi mức đo sau khi đã có kết quả, việc có thể làm hỏng thiết bị, bắt đầu việc lớn, ghi đè bản chốt, cài thêm công cụ. Một chữ “có” gõ trong ô chat không mở được cửa; phải bấm trên thẻ. Mọi thay đổi gỡ

lại được, kể cả do người tự gỡ, theo ba cách: gỡ lần sửa vừa rồi, gỡ một lần sửa cụ thể trong lịch sử, hoặc đưa cả kho về một bản chốt đã đặt tên; việc không gỡ lại được như nạp chip vẫn được ghi kèm lý do.

D. Số nào cũng phải có nguồn

Luật nền thứ nhất của EIDE: mỗi con số dùng để quyết định phải lần về được chỗ nó lấy ra. Mỗi thông số trong kho mang một bậc tin được: VÀNG (tài liệu của hãng, đã có người xác nhận), BẠC (mã đọc thẳng từ trang PDF, chưa ai xem lại), NGƯỜI (người dùng nói, có trích nguyên văn), CÀI ĐẶT (đọc từ tệp cấu hình `.ioc`, `sdkconfig`, `.ld`, `.map`), ĐỒNG (đoán từ hình hoặc từ tên gọi, chỉ để gợi ý). Đường nâng từ BẠC lên VÀNG chỉ người đi được. Con số lấy từ PDF được đọc bằng mã theo trang, không để mô hình đọc hộ; ký hiệu được chuẩn hoá lúc đọc (4R7 thành 4,7 Ω) và TBD thành “chưa có”, không thành 0. Đoạn nào trong tài liệu trông như đang ra lệnh cho tác tử thì được đọc như dữ liệu, không như lệnh.

E. Hai việc ít tác tử khác làm

Thứ nhất, tác tử mở lại tệp nó vừa tạo rồi đếm số đoạn, số bảng, số hình, rồi mới báo; vì câu “ghi thành công” chỉ nói về lời gọi, không nói về tệp trên đĩa. Thứ hai, tác tử tự viết được công cụ mới cho chính nó: khi thiếu công cụ, nó viết một công cụ kèm bộ kiểm và chỉ nạp khi bộ kiểm chạy đúng. Việc này đã chạy thật: 8/8 ca trong bộ thử, ba công cụ mới trong phiên FreeRTOS, bốn công cụ cho FPGA trong hai ngày (Mục IV-C).

F. Giữ kín bằng cấu trúc

Khoá mô hình chỉ nằm trong tệp `.env` không vào git; danh sách mô hình cho phép chỉ có một mục, và trong phiên

robot cả 1 415 lời gọi đều là mô hình đó. Ba lớp chặn là mã, không gọi mô hình, nên không tốn token; trong hai phiên trên bo STM32F469 chúng can thiệp 57 lần trên 1 233 lời gọi công cụ (4,6 %), gồm 27 lần dừng trước khi nạp chip và 4 lần từ chối ghi hằng số không nguồn [5].

III. NHỮNG VIỆC TÁC TỬ LÀM ĐƯỢC

Bảng I xếp 127 công cụ theo việc người cần làm, không theo cách chia mã. Mỗi nhóm có một ràng buộc thi hành bằng mã đi kèm, vì một năng lực không có ràng buộc là một năng lực không tin được.

BẢNG I CÁC NHÓM VIỆC CỦA TÁC TỬ VÀ RÀNG BUỘC ĐI KÈM		
Việc	Số	Tác tử làm được gì — và ràng buộc
Làm rõ đề bài	11	hỏi gộp một cụm câu kèm lý do và giả định nếu bỏ qua; ghi yêu cầu kèm câu nói nguyên văn; nêu 2–4 cách làm rồi để người chốt
Đọc tài liệu	20	nạp PDF, Word, Excel, slide, mã nguồn; nhận dạng loại tệp theo byte đầu; rút hình kèm chữ trong hình; lấy con số bằng mã, trích theo trang/dòng/ô
Viết tài liệu	1	dựng Word, PowerPoint, Excel, PDF từ nguồn Markdown trong kho; từ chối dựng Excel khi nguồn không có bảng
Thiết kế mạch	27	bản đồ mạch; cây khối nhiều cấp; thư viện khối; sinh sơ đồ nguyên lý mở được bằng KiCad; không gán chân không có trong tài liệu
Viết mã	8	đọc hiểu mã cũ trước khi sửa (E4020: không ghi đề tệp chưa đọc hết); dịch cho AVR, ARMv6-M/v7-M/v7E-M, RISC-V; bản đồ bộ nhớ; đo xem bộ kiểm có đo gì không
Chạy thử trên máy	2	nêu mức đo trước, chạy, rồi đối chiếu; bộ mức đo chấm, không phải chương trình
Bo thật	6	dò bo (đọc mã chip từ silicon); nạp qua ST-Link, avrdude; đọc ngược so từng byte; đọc công nối tiếp; giải mã thanh ghi lỗi; đọc khung ảnh từ chip
Chip trên FPGA	5	soát cú pháp Verilog; mô phỏng testbench; tổng hợp; đặt-đi dây và đo Fmax; đóng gói và nạp bitstream
Nhớ và quản việc	28	kho dữ liệu; sổ ghi việc; gỡ lại; bản chốt; nhánh; chia việc lớn thành chặng, mỗi chặng để lại thứ mở ra xem được
Tự lo cho mình	4	tìm công cụ; tự viết công cụ mới rồi tự kiểm trước khi dùng

Tác tử được thử ở bốn mức, vì bốn mức đo bốn thứ khác nhau: ca kiểm đơn vị (1 575 ca Python, 40 ca Swift) đo từng hàm và luật chặn; ca đi qua giao thức thật không tốn tiền mô hình, kèm các đường hồng cổ ý; 76 ca kiểm thuộc 19 kịch bản chạy qua ứng dụng thật bằng một kênh tệp thay ngón tay người, kênh này đọc ngược được trạng thái giao diện; và một việc thật trên bo thật, mức duy nhất trả lời được câu “tác tử có gánh nổi một việc phức tạp không”. Bảng II tóm tắt kết quả đến 04/10/2026.

BẢNG II KẾT QUẢ THỬ ĐẾN NGÀY 04/10/2026	
Phép đo	Kết quả
76 ca kiểm, 19 kịch bản, qua ứng dụng thật	68/68 ca đo được đạt; 8 ca còn lại cần người, cần thiết bị hoặc ngoài phạm vi
Quét 11 tab giao diện	124/124 ô
Ca kiểm đơn vị Python / Swift	1 575 / 40 (13 ca đo khi trả lại mã cũ)
Tác tử tự viết công cụ cho chính nó	8/8
Viết tài liệu Word, PowerPoint, Excel, PDF	13/13
Chia việc lớn rồi ráp lại	12/14
Mã có nói khác tài liệu thiết kế không	0 chỗ lệch trên 15 tệp
Luồng FPGA: Verilog → bitstream → nạp → đo	chạy thông tới silicon, 96/96 ô
Công cụ đã được dùng thật trong sổ ghi việc	115/127

Một phép đo đáng nói riêng: công cụ test . sensitivity cố ý sửa hồng mã sản phẩm rồi xem bộ kiểm có chuyển sang đỏ không, sau đó trả mã về nguyên trạng. Phép đo này cần, vì trong phiên robot, ba lần liên tiếp tác tử viết bộ kiểm cho đúng chỗ nó vừa sửa, và cả ba lần bộ kiểm chỉ chép lại logic sang tệp kiểm rồi so với chính nó, xanh bất kể mã đúng hay sai. Đây đúng là hành vi mà [4] đo được trên bộ bài tập. Sau khi bị bắt viết lại cho bộ kiểm dùng thẳng mã sản phẩm, sáu phép phá đều chuyển đỏ đúng lúc. Một việc khác đáng kể: rả sổ ghi việc phát hiện 31 trong 122 công cụ chưa bao giờ được dùng, bảy công cụ có đường dẫn bị đứt; nay 115/127 đã được dùng thật.

IV. BA VIỆC THẬT TRÊN BA BO MẠCH

Cả ba việc dùng cùng tác tử, cùng mô hình gemini-3.8-flash, và người thao tác là tác giả thứ nhất. Mọi con số dưới đây lấy từ sổ ghi việc, changeset và nhật ký lời gọi mô hình trong kho của từng dự án [6], không lấy từ lời kể của tác tử. Hai việc đầu được làm hai lần; số của lần làm lại từ dự án trống (ngày 03–04/10/2026) được dùng làm số chính.

A. Việc 1: tự viết nhân hệ điều hành thời gian thực thay FreeRTOS

Đề bài: bỏ FreeRTOS, viết lấy nhân thời gian thực cho bo STM32F469I-DISCO (Cortex-M4F, 180 MHz), chạy tới khi màn hình LCD 800×480 và cảm ứng lên đúng như bản cũ, dùng cùng tệp driver của hãng. Bảng III là số đo trên kit của lần làm lại ngày 04/10.

BẢNG III NHÂN THỜI GIAN THỰC TỰ VIẾT, ĐO TRÊN STM32F469I-DISCO	
Chỉ số	Giá trị
Mã tác tử tự viết	1 211 dòng: lập lịch, chuyển ngữ cảnh PendSV bằng hợp ngữ, hàng đợi tĩnh, 6 tác vụ, giao diện 3 trang
Mã lấy của hãng, không sửa	110 676 dòng (HAL, CMSIS, BSP, driver màn hình và cảm ứng)
Ký hiệu FreeRTOS còn trong ảnh	0
Nhịp hệ thống đo thật	≈ 1 003 Hz (đọc uwTick cách

Chỉ số	Giá trị
	nhau 10 s); SWS = PLL, 180 MHz
Mức nước ngăn xếp đo trên chip	LED 19/128, Button 25/128, Monitor 29/128, Display 64/512 word
Lỗi phần cứng	CFSR = 0, HFSR = 0
Giao diện cảm ứng	3 trang, 6 lần đổi trang / 7 lần chậm, đọc từ ô nhớ trên chip
Kích thước ảnh	265 152 B so với bản FreeRTOS 260 204 B (lớn hơn 1,9 %)
Phiên làm việc	25 bước qua giao diện, 837 lời gọi mô hình, 103 changeset, 5,0 giờ

Về kích thước ảnh, điều kiện tự đặt là “không lớn hơn bản cũ (khoảng 263 KB)”: đọc theo số làm tròn thì đạt, đọc theo bản nhị phân thì không. Chỗ mơ hồ này là lỗi câu chữ của người viết đề bài, giữ nguyên vì nó minh hoạ điều ở Mục V: tiêu chí viết lòng thì không đo được gì.

Phát hiện chính của việc này không phải con số hiệu năng mà là một họ lỗi lặp lại: tác tử viết đúng một cơ chế nhưng không nối nó vào hệ, và không lần nào có lỗi báo ra. Trong hai ngày gặp bảy lần: `rtos_tick()` viết đúng nhưng ô vector `SysTick` vẫn trả `Default_Handler`; `PendSV_Handler` nối đúng vector, đúng ưu tiên, nhưng không ai đặt bit `PENDSVSET`; có `RTOS_IDLE_PRIORITY` nhưng không ai tạo tác vụ rồi. Ba lỗi này nối thành chuỗi mà mỗi khâu chỉ lộ ra sau khi vá khâu trước, và cả ba cho cùng một triệu chứng: bo tối. Phần khó nhất (chuyển ngữ cảnh bằng hợp ngữ) tác tử viết đúng ngay; phần nó thua là phần nối.

Điều cứu được thời gian là tác tử tự khai điểm mù của bộ kiểm nó viết. Sau khi bộ kiểm xanh 4/4, nó tự phá mã bốn lần rồi báo hai ca bộ kiểm không bắt được (thứ tự `xPSR` và `PC` trên ngăn xếp; giá trị `EXC_RETURN`), kèm dự đoán “ nạp lên bo sẽ nổ `HardFault` ngay chu kỳ đầu”. Người kiểm lại bằng tay: đổi `EXC_RETURN` thành 0, cả bốn ca vẫn xanh; khi chuyển ngữ cảnh chạy thật, bo nổ đúng `IBUSERR` ở đúng chỗ ấy. Cách làm đáng giữ lại là đo trước, sửa sau: khi hai nút bấm không ăn, thay vì đoán giữa ba nguyên nhân, yêu cầu tác tử thêm ba ô nhớ đọc qua cổng gỡ lỗi (cảm ứng còn sống, đếm được 11 lần chậm, 0 lần đổi trang); một lượt đọc loại trừ hai nguyên nhân bằng con số.

B. Việc 2: robot hai bánh tự cân bằng, làm hai lần

Đề bài: đọc tập tài liệu bàn giao phần cứng của một robot hai bánh (ATmega328P, MPU6050, hai driver bước A4988) rồi viết phần mềm để robot đứng được. Hồ sơ đã cố ý bỏ hẳn phần thuật toán: không có vòng điều khiển, bộ lọc hay PID để chép; chỉ còn các điều kiện bắt buộc về phần cứng, trong đó hàm ngắt phát xung 50 kHz không được có số thực, không có phép chia, không gọi hàm.

Lần 1 (01/10/2026): tác tử viết 1 820 dòng C, 10 mô-đun; hàm ngắt 50 kHz có 35 lệnh, 0 lời gọi, 0 số thực, 0 phép chia; dựng bảng 109 điều kiện bắt buộc từ tài liệu và đối chiếu lại sau mỗi lần sửa; 1 415 lời gọi mô hình; 7,4 giờ; robot đứng được. Báo cáo của việc này nói thẳng tám chỗ tác tử báo xong trong khi đang sai: tắt động cơ bằng một chân đã nối cứng xuống đất; chạy thử báo 6/6 đạt trong khi đã đảo dấu khâu P;

chữ “đạt” viết cứng trong lệnh in; báo nạp xong một bản trong khi trên chip là bản khác (avrdude so với đúng tệp nó được đưa, nên `verified`, 0 byte lệch vẫn xanh); ba bộ kiểm liên tiếp chỉ chép logic. Không chỗ nào do tác tử tự tìm ra; tất cả đều do một phép đo người thiết kế. Robot chỉ đứng sau khi có bản của nhà cung cấp đã chạy được để so: đối chiếu tìm ra bảy chỗ lệch, nặng nhất là hằng số hiệu chuẩn gia tốc 535 thay vì 92, tức điểm cân bằng sai 3,1°.

Lần 2 (04/10/2026): bảy chỗ sai thành bảy bản vá cho EIDE, và đề bài được viết lại tường minh: mốc so sánh do người tự tính tay ghi ở tầng NGƯỜI; nghiệm thu đòi đo tần số ngắt thật thay vì tin giá trị cấu hình; tham số đã chạy thật ghi kèm nguồn. Rồi làm lại từ dự án trống (Bảng IV).

BẢNG IV
HAI PHIÊN ROBOT, CÙNG ĐỀ BÀI

Chỉ số	Lần 1 (01/10)	Lần 2 (04/10)
Mã tác tử tự viết	1 820 dòng	1 124 dòng
Lời gọi mô hình	1 415	670
Lời gọi công cụ bị chặn	26 (6,6 %)*	18 (3,3 %)
Lỗi tác tử do người bắt	8	1
Lỗi người do tác tử bắt	0	3
Thời gian phiên	7,4 giờ	4,53 giờ
Tiền mô hình (mức giữa)	≈ 340 nghìn đồng	≈ 34 nghìn đồng
Robot đứng được	có	có

*: số của phiên 03/10, là lần chạy đầu của đợt làm lại, dừng chủ động ở bước 23/32 sau khi đo ra bảy chỗ sai.

Số đo trên ATmega328P thật ở lần 2: ngắt phát xung 50,0005 kHz (lệch +0,0009 % so với ngưỡng ±1 %), đo bằng tỉ số hai bộ đếm neo vào đồng hồ tường; 0/369 dòng theo dõi bị thiếu; 0 lệnh chia hay số thực trong hàm ngắt theo `objdump`; 372/372 khoảng bản tin đúng 100 ms; 22/22 điều kiện phần chủ đạt; 4/4 phép phá mã bị bộ kiểm bắt.

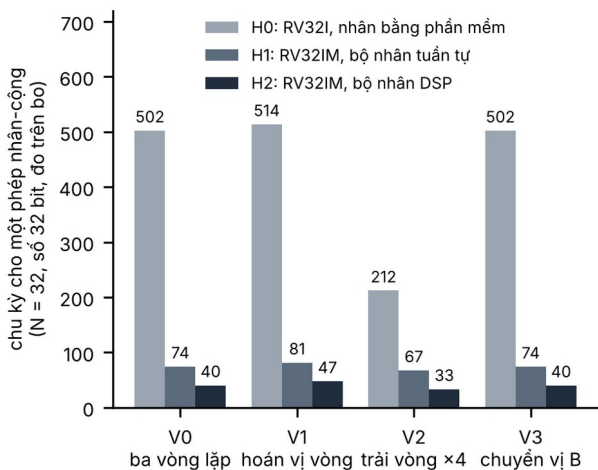
Chỗ đáng kể nhất là chiều bắt lỗi đối. Ở lần 1, mọi lỗi đều do người bắt. Ở lần 2, tác tử bác một kết luận sai của người: người cho rằng vòng 250 Hz đã đo xong bằng `millis()`; tác tử chỉ ra `millis()` phân giải 1 ms nên chu kỳ 4 ms có sai số tức thời tới ±25 %, phép đo chỉ chứng minh trung bình chứ không chứng minh các nhịp cách đều; nó cũng tìm ra lỗi đọc rách một biến 32 bit trên CPU 8 bit. Nhưng lần 2 rẻ hơn 10 lần không phải vì mô hình giỏi lên, mà vì phụ lục đã trao sẵn bộ tham số đã chạy thật, trong đó có đúng con số 92 mà lần 1 ghi sai; nó đo giá trị của việc tài liệu hoá phần khó, không đo mô hình.

C. Việc 3: lỗi RISC-V trên FPGA, 96 ô đo trên silicon

Việc này cố ý nằm ngoài hẳn vùng EIDE từng làm: trước đó EIDE không có một dòng nào về HDL, không tổng hợp hay mô phỏng Verilog được, không nạp FPGA được, không dịch cho RISC-V được. Đề bài: dựng một CPU RISC-V (lời mở PicoRV32 [7]) trên kit Sipeed Tang Nano 20K (Gowin GW2AR-18), chạy chương trình C trên CPU đó, rồi đo chi phí nhân ma trận ở ba cấu hình phần cứng. Trong hai ngày 03–04/10, chuỗi công cụ mã mở được dựng (Yosys, nextpnr, Apicula, openFPGALoader, `riscv64-unknown-elf-gcc`) và bốn công cụ EIDE được viết thêm (nạp FPGA, đọc mã chip, bắt bản ghi quanh lúc nạp, chốt phiên khi gói giao diện cũ hơn mã).

Bài 1, CPU sống: SoC gồm PicoRV32 RV32I, 32 KB BRAM, UART 115 200 baud, LED; mã khởi động, linker script và main.c do tác tử viết. Trên bo, chuỗi Hello from PicoRV32 on Tang Nano 20K, cycle=... in ra mỗi giây; hiệu số cycle giữa các dòng là 27 000 001, 27 000 031, 27 000 024, đúng xung nhịp 27 MHz. Nguyên nhân gốc mất gần một ngày: chân 88 đọc mức 0 khi không ai bấm, giữ reset mãi. Tác tử tự khoanh được bằng bộ phát chẩn đoán nó viết, rồi ghi quyết định bỏ nút khỏi mạch reset kèm mặt dờ.

Bài 2, 96 ô đo: 4 kích thước ma trận ($N = 4, 8, 16, 32$) $\times$ 2 kiểu dữ liệu (32 bit, 8 bit) $\times$ 4 cách viết (ba vòng lặp, hoán vị vòng, trái vòng, chuyển vị) $\times$ 3 cấu hình CPU (H0 không bộ nhân; H1 bộ nhân tuần tự; H2 bộ nhân nhanh dùng DSP). Mốc so sánh là bốn tổng kiểm người tự tính tay, độc lập với mã của tác tử, chốt vào đề bài ở tầng NGƯỜI trước khi đo. Bốn phép kiểm do người tự làm trên bản ghi gốc: 96/96 ô bắt được từ cổng nối tiếp; 96/96 ô ok = 1; 0 ô lệch tổng kiểm so với bảng tính tay; 0 ô lệch số chu kỳ giữa bo thật và mô phỏng. Lệch 0 % có lý do: lỗi tuần tự, bộ nhớ chỉ có BRAM nội với trẻ cổ định, nên mô phỏng từng sườn xung nhịp của chính mã RTL cho đúng số. Biến độc lập kiểm được trong mã máy: cấu hình H0 còn 24 lời gọi __mulsi3/__udivdi3, hai cấu hình kia còn 0.



Hình 3. Chu kỳ cho một phép nhân-cộng ở $N = 32$, đo trên bo Tang Nano 20K: bộ nhân tuần tự giảm 502 $\rightarrow$ 74, bộ nhân DSP giảm tiếp xuống 40.

Hình 3 cho một lát cắt. Trên cả 96 ô, bộ nhân tuần tự nhanh hơn nhân bằng phần mềm từ 1,57 đến 6,8 lần; bộ nhân DSP từ 2,45 đến 12,5 lần. Đáng kể hơn con số lớn nhất là chỗ con số nhỏ nhất: cả hai ô tăng ít nhất đều là số 8 bit với cách viết hoán vị vòng, nơi cấu hình không có bộ nhân đã nhanh sẵn (131 thay vì 502 chu kỳ). Tác tử giải thích bằng định luật Amdahl: khi phần thời gian dành cho phép nhân đã nhỏ, thêm bộ nhân phần cứng chỉ cải thiện được chút ít. Câu trả lời cho “lúc nào thêm phần cứng không giúp gì” chỉ hiện ra khi đo đủ 96 ô.

TÁC TỬ [G-FLASH] Nạp gì (hash, kích thước) $\rightarrow$ vào đâu; đã có snapshot chưa

- Tác động lên phần cứng thật: flash.
- Chưa có bản ứng ý nào để quay về nếu việc này hỏng.
- Thao tác này KHÔNG hoàn tác được.

$\rightarrow$ Duyệt · Từ chối

BẠN Duyệt cổng (gate-0024) — vì: đồng ý, đây là việc mình vừa nhà

BẠN Đủ 96 ô từ bo thật. Minh bắt lại H1 bằng vòng đọc đã sửa, và giữ cả ba bản ghi đều có DONE_BENCHMARK cùng 32 dòng đủ trường. Minh tự đối chiếu:

phép kiểm	kết quả
ô bắt được từ bo	96/96
ô ok=1	96/96
ô có tổng kiểm lệch bằng tay NGƯỜI mục 5.3b	0

Ba tệp: bai2/ket-qua/bo-that-h0.log, -h1.log, -h2.log.

Hình 4. Một đoạn màn hình EIDE trong phiên FPGA: thẻ của G-FLASH dùng lượt trước khi nạp, người bấm Duyệt, rồi tác tử đối chiếu 96/96 ô với bảng tính tay ở tầng NGƯỜI.

Tác tử tự viết 2 020 dòng (Verilog, C, hợp ngữ, linker script, Python, ràng buộc chân), qua 54 bước có ảnh của số, 1 307 lời gọi mô hình, 161 changeset, 16 789 dòng số ghi việc. Sáu lỗi của tác tử đều bị bắt bằng cách mở mã hoặc mở số ra đối chiếu, không bằng đọc báo cáo của nó: cờ ok của cách viết cơ bản tự so với chính nó nên 24 trong 96 ô không thể báo sai; bảng tổng kiểm chuẩn sinh ra nhưng main.c không #include; cách viết V3 đối thuật toán mà không báo; chú thích nói vùng đệm 12 KB, mã khai 16 KB; Makefile thiếu -lgcc; và nguy hiểm nhất, mô phỏng và bitstream nạp hai tệp dịch khác nhau (-Os và -O2) rồi đem so với tiêu chí “bo lệch mô phỏng $\leq 1\%$ ”. Hai lỗi khác tác tử tự tìm và tự báo đúng loại: lệnh chia bẫy CPU vì phần cứng để ENABLE_DIV = 0, và hàm memset tự viết bị trình dịch đổi thành lời gọi chính nó. Ba lỗi của người cũng được giữ lại vì chúng đo chất lượng đề bài: tiêu chí “mọi ô ok = 1” tự nó vô hiệu vì cờ ấy chỉ so bốn cách viết với nhau, bo tính sai toàn bộ vẫn đạt (đã vá bằng bốn tổng kiểm ở tầng NGƯỜI); đặt tên tệp bản ghi theo cấu hình mình tưởng đang đo, dè mất 22 ô; vòng đọc cổng thoát sớm nên cắt giữa dòng thứ 32.

V. ĐẶT BA VIỆC CẠNH NHAU

BẢNG V
BA VIỆC THẬT: SỐ ĐO CỦA PHIÊN TÁC TỬ

	Nhân RTOS	Robot (lần 2)	RISC-V/FPGA
Bo	STM32F469I-DISCO	ATmega328P	Tang Nano 20K
Mã tự viết	1 211 dòng	1 124 dòng	2 020 dòng
Thời gian phiên	5,0 giờ	4,53 giờ	$\approx$ 14,5 giờ
Lời gọi mô hình	837	670	1 307
Changeset	103	—	161
Bước qua giao diện	25	25	54
Chạy thật trên bo	LCD + cảm ứng, 6 tác vụ, 1 003 Hz	đứng được, 50,0005 kHz	96/96 ô, 0 % lệch mô phỏng

A. Tác tử giúp được ở đâu

Thứ nhất, nó không mỏi, nên không bỏ bước: 96 ô đo, mỗi ô ba lượt và một tổng kiểm; 28 lần dựng bitstream, 80 lượt mô phỏng; mức nước ngăn xếp của cả sáu tác vụ; 109 điều kiện đối chiếu lại sau mỗi lần sửa; 103 và 161 changeset đều gỡ lại

được. Người làm tay tới ô thứ ba mươi thường bắt đầu bỏ phần tổng kiểm; mỗi là lý do người bỏ bước, và nó không mới.

Thứ hai, nó đi được vào vùng chưa có công cụ: việc FPGA bắt đầu từ chỗ EIDE không biết gì về HDL, và trong hai ngày kết quả đi tới silicon nhờ khả năng tự viết công cụ rồi tự kiểm trước khi dùng.

Thứ ba, nó tự khai điểm mù và nhận sai trước khi bị truy: hai điểm mù của bộ kiểm RTOS khai trước rồi nổ đúng chỗ; khi được hỏi, nó nhận “đã phạm đúng lỗi này ở lượt vừa rồi: chỉ viết hai hàm khung rỗng để dịch qua”; khi viết thêm 36 dòng mã không ai gọi chỉ để một tiêu chí được thoả, nó ghi cả động cơ vào chú thích, nhờ đó 36 dòng ấy không lọt vào con số “tự viết”.

B. Tác tử không giúp được ở đâu

Thứ nhất, nó không tự biết nó sai: mọi câu “đã xong, đã kiểm, 0 byte lệch” đều có thể đúng về lời gọi và sai về kết quả; tám chỗ ở phiên robot lần 1 và sáu chỗ ở phiên FPGA đều cần một phép đo người thiết kế. Thứ hai, nó yếu ở phần nói: bảy lần trong hai ngày viết đúng một cơ chế rồi không gọi tới nó, không lần nào có lỗi báo ra; nó cũng hay mô tả phép đo thay vì chạy phép đo, như nói “nhịp 1 000 Hz” theo giá trị nạp vào SysTick trong khi nhịp thật là 91 Hz vì chip đang chạy 16 MHz thay vì 180 MHz. Thứ ba, nó không chạm được vào vật thật: mỗi lần robot ngã cần người dựng lên, mỗi lần bo tối cần người đọc thanh ghi qua SWD. Vì vậy mức lợi của tác tử tỉ lệ với phần việc nằm trong máy tính: việc nhân RTOS gần như toàn bộ là đọc tài liệu, tra thanh ghi, viết mã nên khoảng chênh lớn; việc robot phần lớn thời gian nằm ở gỡ lỗi trên bo nên khoảng chênh hẹp hơn nhiều.

C. Thời gian và tiền, và điều bảng này không chứng minh

Với hai việc có ước lượng, một đội người làm tay (1 Senior, 1 Mid, QA, quản lý; ước theo PERT ba mức cho 13 việc nhỏ, kiểm chéo bằng COCOMO làm mức trên) cần khoảng 31–32 ngày công, tức 4 tuần lịch, và khoảng 105–112 triệu đồng; phiên tác tử tốn 4,5–7,4 giờ của một người và 34–340 nghìn đồng tiền mô hình, cho cùng kết quả robot đứng được. Bốn điều bảng này không chứng minh, nói trước để không bị đọc quá tay: (1) người vẫn nằm trên đường quyết định, phần lớn thời gian phiên là chờ người thử trên bo; (2) bước cuối của lần 1 dùng một bản đã chạy được để so, không có nó thì phiên còn kéo dài; (3) tiền mô hình không phải toàn bộ chi phí, chưa tính giờ người, phần cứng và thời gian dựng EIDE; (4) các chỗ tác tử báo xong mà sai đều cần một người biết phải nghi chỗ nào; giao cùng việc cho người không biết đặt phép đo thì các chỗ ấy lọt hết, và số tiền đó mua được một phần mềm trông như đã xong.

Điều rút ra lớn hơn con số: chất lượng đề bài quyết định chất lượng kết quả nhiều hơn dự tính. Ba tiêu chí người viết lóng đã sinh ra 24 ô đo không thể báo sai, 36 dòng mã không ai gọi, và một bản nạp chạy chậm 11 lần mà không ai biết; tác tử làm đúng theo tiêu chí sai, nhanh và triệt để. Cách sửa đã hiệu nghiệm ở lần 2 của robot: mốc so sánh do người tính trước ghi ở tầng NGƯỜI; đòi đo, không tin giá trị cấu hình; số dòng là thông tin, không phải chỉ tiêu.

VI. HẠN CHẾ

Ba việc, một người thao tác, một mô hình; đổi mô hình có thể đổi hành vi, nên cần đo lại. Cột người làm tay là ước lượng của chính nhóm tác giả, không phải số đo. Lần 2 của robot

được trao sẵn tham số đã chạy thật, nên khoảng chênh giữa hai lần không đo năng lực mô hình. Việc FPGA chưa dựng lại được bằng make ngoài EIDE vì đường dẫn include tính từ gốc dự án; 96 ô là thật nhưng người khác gỡ lại chưa ra. Tác tử chưa có công cụ chạy make hay chạy tệp Python, nên hai lần phải nhờ người. 12 trong 127 công cụ chưa được dùng thật. Robot đã tháo nên “robot đứng được” là quan sát của người, chưa có phân bố góc kèm theo.

VII. KẾT LUẬN

Bài báo trình bày một tác tử lập trình nhúng với kiến trúc vòng lặp bao bởi ba lớp chặn bằng mã, 127 công cụ, 11 cửa duyệt và số ghi việc móc bầm, rồi đo nó trên ba việc thật: nhân hệ điều hành thời gian thực tự viết chạy trên STM32F469 với nhịp 1 003 Hz, robot hai bánh đứng được trên ATmega328P với ngắt 50,0005 kHz, và lõi RISC-V trên FPGA với 96 ô đo trên silicon lệch 0 % so với mô phỏng. Hình dung đúng không phải “tác tử thay người”, mà là: tác tử gánh phần ghi chép và phần làm đủ bước, người giữ phần phán đoán và phần chạm vào vật thật; và có một vai thứ ba hoá ra cũng quan trọng, là một bản đã chạy được dùng để so sánh. Tác tử chỉ tin được khi nghiệm thu nằm ngoài tay nó: mốc do người tính trước, tiêu chí nêu trước khi chạy, và bộ kiểm phải chứng minh được rằng nó đo khi mã hồng. Hướng tiếp theo là đo lặp trên nhiều mô hình và nhiều người thao tác, và thêm công cụ chạy lệnh dựng cho tác tử.

TÀI LIỆU THAM KHẢO

- [1] Thủ tướng Chính phủ, “Quyết định số 21/2026/QĐ-TTg ban hành Danh mục công nghệ chiến lược và Danh mục sản phẩm công nghệ chiến lược,” 30/4/2026. [Trực tuyến]. <https://congbao.chinhphu.vn/van-ban/quyet-dinh-so-21-2026-qd-ttg-469478.htm>
- [2] Z. Englhardt, R. Li, D. Nissanka, Z. Zhang, G. Narayanswamy, J. Breda, X. Liu, S. Patel, và V. Iyer, “Exploring and characterizing large language models for embedded system development and debugging,” trong Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, ACM, 2024. arXiv:2307.03817.
- [3] Y. Li và cộng sự, “Skilled AI agents for embedded and IoT systems development,” arXiv:2603.19583, 2026.
- [4] Z. Zhong, A. Raghunathan, và N. Carlini, “ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases,” arXiv:2510.20270, 2025.
- [5] Vũ Trí Công và Nguyễn Trung Hiếu, “Kiến trúc tác tử hỗ trợ lập trình nhúng: từ định tuyến ý định đến vòng lặp có kiểm soát,” bản thảo gửi Hội nghị REV-ECIT 2026.
- [6] Vũ Trí Công và Nguyễn Trung Hiếu, “EIDE v3: lõi tác tử kỹ sư nhúng; mã nguồn, số ghi việc và nhật ký ba việc thật (thư mục docs/rtos-tu-viet, docs/robot-sinhvien, docs/fpga),” 2026. [Trực tuyến]. <https://github.com/mobiluckvn/EIDEv3>
- [7] C. Wolf, “PicoRV32: a size-optimized RISC-V CPU,” mã nguồn mở. [Trực tuyến]. <https://github.com/YosysHQ/picorv32>